

RAG 第一阶段：文档索引与向量化

现在开始开发 RAG 项目的第一阶段。

严格按照项目总提示词执行。

本阶段只实现“文档 → Vectorize”的 Indexing Pipeline。

不要实现第二阶段的 Query Retrieval。

不要实现第三阶段的 Gemini RAG Answer。

一、本阶段目标

完成：

文件上传 → 文件存储 → 文件解析 → 文本清洗 → Chunk → Gemini Embedding → Cloudflare Vectorize

最终我要能够确认：

“一个真实上传的文档，已经被切成多个 Chunk，并且每个 Chunk 已经生成 Embedding 并成功写入 Vectorize。”

二、前端功能

完成基础 UI。

页面采用左右两栏布局。

左侧：

- 上传文件按钮
- 文件列表
- 文件名称
- 文件类型
- 文件大小
- 文件处理状态
- 删除按钮
- 当前处理中的状态展示

右侧：

暂时不需要完整 AI 对话功能。

可以保留 AI 对话区域的基础 UI，但本阶段不要实现真正的 AI 问答。

右侧底部可以保留输入框，但发送功能暂时禁用，并明确标记当前阶段尚未启用问答。

三、上传流程

用户选择文件后：

前端：

POST /api/documents

Worker：

1. 验证文件。
 2. 生成 documentId。
 3. 将原始文件保存到 R2。
 4. 根据文件类型调用 Parser。
 5. 得到纯文本。
 6. 清洗文本。
 7. Chunk。
 8. 调用 Gemini Embedding。
 9. 将 Vector + metadata 写入 Vectorize。
 10. 返回处理结果。
-

四、Parser

设计统一接口：

parseDocument(file)

根据文件类型调用不同解析逻辑。

至少支持：

TXT

Markdown

PDF

如果 PDF 在当前 Worker Runtime 中存在限制：

优先选择兼容 Workers 的方案。

不要引入需要完整 Node.js Runtime 才能运行的重量级 PDF 依赖。

如果 PDF 暂时无法可靠运行：

先保证 TXT / Markdown Pipeline 完整运行，并将 PDF Parser 封装成独立模块，方便后续替换。

五、文本清洗

实现基础 text cleaning：

- 去除无意义空白
- 合并连续空行
- 规范换行
- 去除明显无意义字符

不要做复杂 NLP。

六、Chunk

自己实现 Chunker。

不要使用黑盒 RAG Chunking 服务。

支持：

chunkSize

chunkOverlap

默认参数设置合理。

Chunk 必须保存：

- chunkId
- documentId
- filename
- chunkIndex
- text

Chunk 应该保证：

1. 不超过指定 chunkSize 太多。
2. 相邻 Chunk 保留 overlap。
3. chunkIndex 连续。
4. 空文本不生成 Chunk。

代码要容易理解。

七、Embedding

创建独立：

services/embedding.js

负责：

text → Gemini Embedding API → vector

不要把 Gemini API 请求代码散落到各个文件。

API Key 使用 Worker Secret。

禁止暴露给前端。

八、Vectorize

创建独立 Vectorize Service。

负责：

upsert vectors

每个 Vector：

- id
- values
- metadata

metadata 至少包含：

documentId

filename

chunkIndex

text

如果 metadata 有大小限制：

不要强行保存过大的字段。

可以根据 Vectorize 实际限制进行合理设计。

九、R2

R2 保存原始文件。

建议：

`documents/{documentId}/{filename}`

保存：

- 原始文件
- content type
- 必要 metadata

R2 是原始文件存储。

Vectorize 是向量检索。

两者职责不要混淆。

十、文档状态

前端至少能够看到：

- waiting
- processing
- completed
- failed

如果处理失败：

必须返回可读错误。

不要吞掉错误。

十一、API

至少设计：

POST `/api/documents`

GET `/api/documents`

DELETE `/api/documents/:id`

具体 API 设计可以根据项目现有结构调整。

GET 用于获取文件列表。

DELETE 必须同时考虑：

R2 原文件

以及 Vectorize 中属于该 documentId 的 vectors。

如果当前 Vectorize 删除需要先查询 vector IDs：

实现合理的删除流程。

十二、参数设置

左侧 UI 增加：

Chunk Size

Chunk Overlap

默认：

chunkSize = 800

chunkOverlap = 100

如果你认为根据实际 Embedding 模型需要调整默认值，可以调整，但必须说明原因。

这些参数用于上传处理。

不要让参数配置变得复杂。

十三、本阶段测试

完成后必须验证：

1. 上传 TXT。
2. TXT 能被解析。
3. TXT 被 Chunk。
4. Chunk 数量正确。
5. Gemini Embedding 成功。
6. Vectorize upsert 成功。
7. 文件列表出现 completed。

8. 删除文件后 R2 文件删除。
9. 删除文件后对应 vectors 删除。
10. 上传 Markdown。
11. 尽可能验证 PDF。

开发完成后不要进入第二阶段。

请告诉我：

1. 修改了哪些文件。
2. 新增了哪些模块。
3. API 如何调用。
4. 如何配置 Gemini Secret。
5. 如何配置 R2。
6. 如何配置 Vectorize。
7. 如何本地运行。
8. 如何测试。
9. 本阶段 RAG 数据流是什么。
10. 当前还有哪些限制。

最重要：

本阶段结束后停止。